

Now add verification loops. Combine an LLM ‘judge’ with deterministic checks: schema validation, unit conversions, rule engines, and fact verification against sources. Monitor hallucination at the operational level with groundedness metrics, citation precision/recall, and acceptance tests per task. Red-team with adversarial prompts and prompt-injection tests (especially in RAG) to stop ‘retrieval hijacking’. In real systems, hallucination management is not a ‘set it and forget it’ affair – it is an observability issue that uses prompts, retrieved contexts, model versions and outputs so that failures can be reproduced, measured, and systematically reduced.

Key considerations for choosing the right LLM

Choosing an open source LLM is an engineering decision across its ability, cost, deployment constraints, and governance. Begin with the workload archetype: conversational support, document understanding, code generation, extraction into structured fields, summarisation, or agentic tool use. Model selection should be based on quantifiable requirements such as minimum accuracy, latency, context length, multilingual support, and acceptable risk, not just on the number of parameters.

Model capability and fit: Test models on domain-specific benchmarks and internal datasets. Overall benchmarks are nice, but the real success to production are your own distributions: ticket text, clinical notes, finance narratives, or policy documents. Evaluate instruction following, long-context performance (including deterioration patterns), and tool-use reliability. For enterprise workflows, choose models that have high structured output fidelity and stable refusal/uncertainty behaviours.

Licensing and compliance:

Open source is not always ‘free and

unrestricted’. Check the licence for rights to use commercially, redistribute, and derivatives finetunes. Verify data handling considerations -- for PII/PHI, favour on-prem or VPC deployment, encryption, access controls and audit logging. Ensure you can address retention, residency and model governance (versioning, reproducibility, rollback).

Compute, latency, and total cost

of ownership: Tune inference — GPU memory, throughput (tokens/sec), concurrency, and peak loads. Quantization is important: for example, there are additional gains (8bit/4bit) from newer serving stacks (vLLM/TGI-like runtimes) that can significantly reduce the cost, but may reduce quality for reasoning or multilingual workloads. Account for context window cost: long-context models raise memory and latency; RAG may be cheaper than ‘pressing the prompt’. Determine if you need full fine-tuning, parameter-efficient tuning (LoRA/QLoRA), or prompt/RAG-only adaptation. In rapid evolving domains, RAG+lightweight adapters tend to be more maintainable than heavy finetunes. Design a data strategy: curated instruction data, preference data for alignment, and evaluation sets

with well-defined criteria.

Safety, robustness, and security:

Evaluate for hallucination rate, toxicity, data leakage, and prompt injection—especially if the model ingests user-provided documents or tickets. Enjoy models with well-established safety tuning and a strong community. Establish guardrails: allow listed tools, policy engines, and human approval for high-stakes actions.

Ecosystem maturity: Evaluate the level of community engagement, the quality of the documentation, whether there are checkpoints, and if it is compatible with your stack (PyTorch/HF, deployment runtimes, monitoring). The ‘right’ LLM is the one you can run reliably — with measurable quality, controlled risk, and predictable cost — throughout its life, not just on a demo.

With ongoing improvements in fine-tuning methodologies, reliability validation, and data protection protocols, open source LLMs are poised to drive responsible and impactful adoption across industries. As the community continues to address challenges related to scalability, bias, and security, the future of generative AI is set to become even more inclusive, transparent, and transformative. 

References

- <https://www.ibm.com/think/topics/open-source-llms>
- <https://github.com/eugeneyan/open-llms>
- <https://www.civo.com/blog/open-source-vs-proprietary-llms>
- <https://www.datacamp.com/tutorial/fine-tuning-large-language-models>
- <https://medium.com/data-scientists-from-future/fine-tuning-open-source-language-models-a-step-by-step-guide-a38bed8df923>
- <https://www.ibm.com/think/topics/ai-hallucinations>
- <https://www.appsmith.com/blog/de-hallucinate-ai-agents>
- <https://www.evidentlyai.com/blog/llm-hallucination-examples>
- <https://www.freecodecamp.org/news/how-to-evaluate-and-select-the-right-llm-for-your-genai-application/>
- <https://www.dataiku.com/stories/blog/key-criteria-when-selecting-an-llm>

By: Dr Magesh Kasthuri and Dr Anand Nayyar

Dr Magesh Kasthuri is a PhD in artificial intelligence and the genetic algorithm. He currently works as a distinguished member of the technical staff (master) and chief architect at Wipro Ltd. This article expresses his views and not of the organisation he works in.

Dr Anand Nayyar is a PhD in wireless sensor networks and swarm intelligence. He works at Duy Tan University, Vietnam, and loves to explore open source technologies, IoT, cloud computing, deep learning, and cyber security.